



Objective

Build a Spring Boot REST service that evaluates loan applications and determines whether a single loan offer (based on requested tenure) can be approved.

Business Context

You are building a backend service for a digital lending platform.

- Receives loan applications
- Evaluates eligibility using business rules
- Generates a single offer based on requested tenure
- Stores decisions for audit

1. Create Loan Application

Endpoint:

POST /applications

Request Body:

```
{
  "applicant": {
    "name": "string",
    "age": 30,
    "monthlyIncome": 75000,
    "employmentType": "SALARIED | SELF_EMPLOYED",
    "creditScore": 720
  },
  "loan": {
    "amount": 500000,
    "tenureMonths": 36,
    "purpose": "PERSONAL | HOME | AUTO"
  }
}
```



2. Validation Rules

- Age must be between 21-60
- Credit score must be between 300-900
- Loan amount must be between 10,000 - 50,00,000
- Tenure must be between 6-360 months
- Monthly income must be greater than 0

Invalid requests must return HTTP 400 with meaningful error messages.

3. Eligibility Rules

- Reject if credit score < 600
- Reject if age + tenure (in years) > 65
- Reject if EMI > 60% of monthly income

EMI Formula:

$$EMI = P * r * (1+r)^n / ((1+r)^n - 1)$$

Base Interest Rate = 12% annually.

Use **BigDecimal**, scale = 2, RoundingMode = HALF_UP.

4. Risk Band Classification

Credit Score	Risk Band
750+	LOW
650-749	MEDIUM
600-649	HIGH



5. Interest Rate Calculation

Final Interest Rate = Base Rate + Risk Premium + Employment Premium + Loan Size Premium

Risk Premium

- LOW → +0%
- MEDIUM → +1.5%
- HIGH → +3%

Employment Premium

- SALARIED → +0%
- SELF_EMPLOYED → +1%

Loan Size Premium

- Loan > 10,00,000 → +0.5%
- Otherwise → +0%

6. Offer Generation

Generate only one offer using the requested tenure.

- Offer is valid only if EMI ≤ 50% of monthly income
- If EMI > 50%, application must be REJECTED

7. Response Format

Approved

```
{
  "applicationId": "UUID",
  "status": "APPROVED",
  "riskBand": "LOW | MEDIUM | HIGH",
  "offer": {
    "interestRate": 13.5,
    "tenureMonths": 36,
    "emi": 16234.23,
    "totalPayable": 584432.23
  }
}
```



Rejected

```
{
  "applicationId": "UUID",
  "status": "REJECTED",
  "riskBand": null,
  "rejectionReasons": [
    "EMI_EXCEEDS_60_PERCENT",
    "AGE_TENURE_LIMIT_EXCEEDED"
  ]
}
```

Technical Expectations

- Spring Boot (latest stable)
- Layered architecture (Controller, Service, Domain, Repository)
- Proper validation and exception handling
- BigDecimal usage for financial calculations
- Unit tests for EMI calculation, risk classification, and eligibility logic

Deliverables

- Version Control: Submission must be a Git repository with full commit history. Do not squash commits before submission.
- Incremental Commits: Commits should reflect logical development steps (e.g., setup, model creation, business logic, tests, refactoring). Avoid single-commit or bulk-dump submissions.
- Commit Messages: Use clear, meaningful commit messages in imperative style. Avoid vague messages such as "update" or "fix".
- Commit Size: Each commit should represent a focused, reviewable change. Do not mix unrelated changes (e.g., refactor + feature + formatting) in a single commit.
- Development Notes: Include a DEVELOPMENT_NOTES.md file explaining:
 - Overall approach
 - Key design decisions
 - Trade-offs considered
 - Assumptions made
 - Improvements you would make with more time
- Refactoring Evidence: Natural iteration is expected. Renaming, restructuring, and incremental improvements are encouraged and will be considered positively